Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 6

з дисципліни «Основи розробки розподіленого програмного забезпечення мовою C++»

Тема: «Клієнт-серверна архітектура додатку. Метод комунікації: відображення (проєкція) файлу в пам’ять»

Виконали:
студенти групи ІС-41
Авдєєв Єгор
Гаврилюк Артем

Перевірив:
Фещенко Кирило Юрійович

Київ 2026

Мета роботи

Набуття практичних умінь у створенні програмних додатків за архітектурою «клієнт-сервер» із використанням відображення файлу в пам’ять (file mapping) для обміну даними між процесами.

Завдання роботи

Розробити програмний комплекс із сервера та двох клієнтів. Сервер повинен створювати проєкцію, читати й відображати дані та закривати її. Клієнти повинні відкривати спільну пам’ять, готувати та записувати дані. Додатково реалізувати автоматичне виявлення змін із використанням події та синхронізацію доступу м’ютексом.

Індивідуальне завдання (Варіант 1)

Номер варіанта обчислено як абсолютну різницю варіантів лабораторних робіт про сокети та поштові скриньки: |4 - 5| = 1. Для виконання використано рядок 1 таблиці завдань лабораторної роботи №4.

Клієнт 1 передає висоту меню (SM_CYMENU), наявність миші (SM_MOUSEPRESENT) та ширину екрана (HORZRES). Клієнт 2 передає ширину й висоту іконки (SM_CXICON, SM_CYICON), ширину вертикальної смуги прокрутки (SM_CXVSCROLL) та глибину кольору (BITSPIXEL).

У таблиці параметр SM_CYMENU підписано як ширину меню, хоча він визначає його висоту. У програмі використано задану константу з правильним підписом «Висота меню».

Теоретичні відомості

File mapping надає процесам доступ до спільної області пам’яті. CreateFileMappingW створює об’єкт, OpenFileMappingW відкриває його за іменем, а MapViewOfFile відображає його в адресний простір процесу. У цій роботі використано пам’ять без окремого файлу на диску. М’ютекс захищає читання та запис, а подія сповіщає сервер про зміни.

Хід роботи

Організація програмного комплексу

Рішення Lab6.slnx містить три проєкти: сервер і два клієнти. Вони збираються у Server.exe, Client1.exe та Client2.exe. Код інтерфейсу розміщено в Lab6.cpp, обмін даними реалізовано в Mapping.h. Роль програми задається константою CLIENT_ID під час компіляції.

Робота сервера

Кнопка «Створити проєкцію» створює іменований об’єкт Local\Lab6_Variant1_Memory, м’ютекс і подію. У спільній структурі SharedData зберігаються ознака роботи сервера, два лічильники змін і два текстові буфери по 512 символів wchar_t. Кожен клієнт записує дані у власний буфер.

Кнопка «Прочитати» вручну отримує копію спільних даних під захистом м’ютекса та показує їх у вікні. «Закрити проєкцію» скидає ознаку активності, викликає UnmapViewOfFile і закриває дескриптори через CloseHandle. Повторне створення на тому самому сервері не відкриває зайвих дескрипторів.

Робота клієнтів

Після натискання «Надіслати дані» клієнт оновлює системні характеристики через GetSystemMetrics і GetDeviceCaps. Далі відкриває проєкцію, захоплює м’ютекс, перевіряє активність сервера, записує текст і збільшує свій лічильник змін. Після SetEvent клієнт звільняє м’ютекс та закриває власні ресурси.

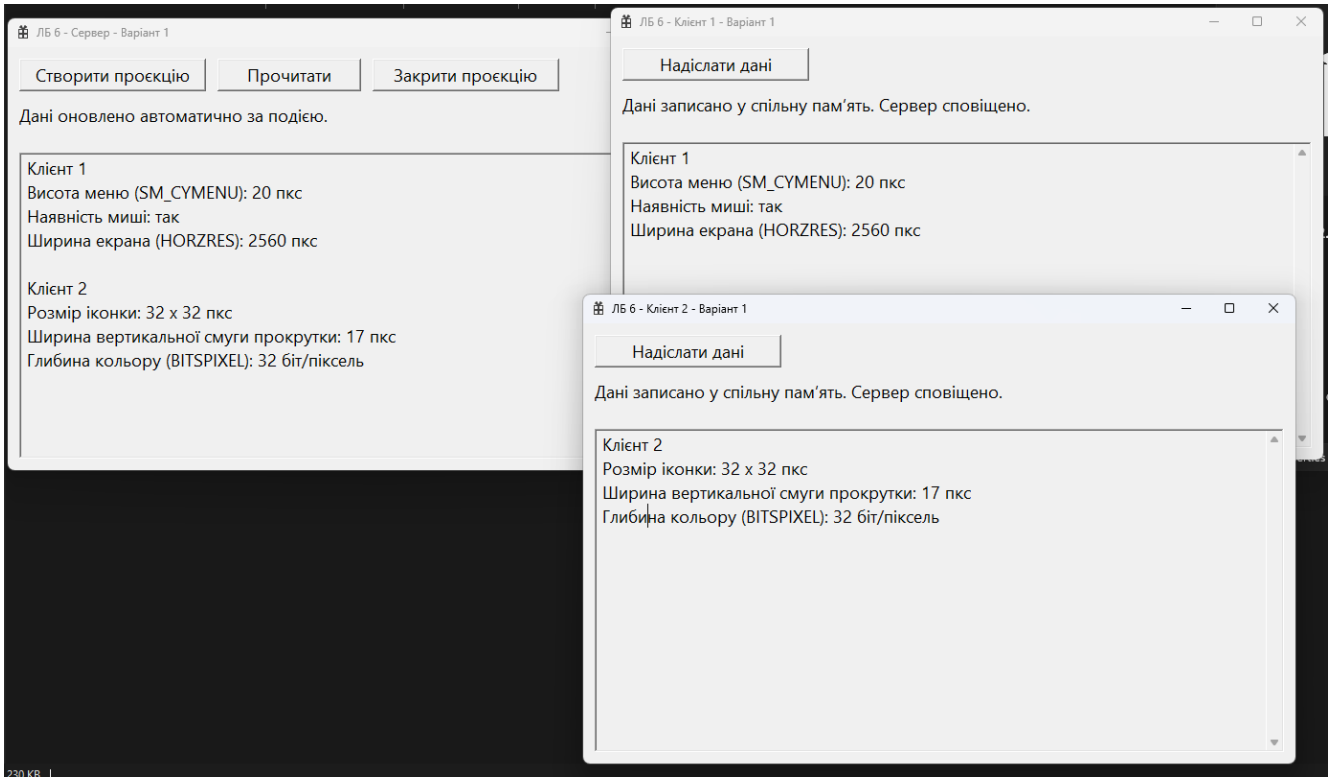
Успішне надсилання означає запис у спільну пам’ять і сповіщення сервера. Окреме підтвердження прочитання не передається. Якщо проєкції немає або сервер зупинено, клієнт показує повідомлення про невдале надсилання.

Автоматичне оновлення реалізовано через MsgWaitForMultipleObjects: сервер очікує подію зміни даних або повідомлення інтерфейсу. Після сигналу він порівнює лічильники змін і відображає актуальні дані. Таймер і окремий робочий потік не потрібні; кнопка «Прочитати» залишається для ручної перевірки.

Перевірка роботи

Програми зібрано у конфігурації Debug / x64. Механізм обміну перевірено тестом verify.cpp: створення, відкриття, передавання українського тексту, спрацювання події, одночасні записи двох процесів, закриття та повторний запуск.

Результати виконання



Усі автоматичні перевірки механізму обміну завершилися успішно. До створення сервера відкрити проєкцію неможливо. Після її створення клієнт записав український текст, сервер прочитав його без змін, а подія перейшла в сигнальний стан і автоматично скинулася після очікування.

Для перевірки одночасного доступу запущено два окремі клієнтські процеси. Кожен виконав по 100 записів. Після завершення перевірено обидва кінцеві повідомлення та лічильники: 101 для першого клієнта з урахуванням попереднього запису українського тексту і 100 для другого.

Після зупинення сервера запис через раніше відкритий клієнтський дескриптор відхилено. Після закриття останнього дескриптора проєкція більше не відкривається. Повторне створення обнуляє лічильники, а нове надсилання успішно читається сервером.

Також перевірено відмову при запуску другого сервера з тим самим іменем проєкції та відхилення повідомлення, довшого за місткість буфера. Повторне закриття не спричинило помилки.

Проєкція зберігає останнє повідомлення кожного клієнта, а не чергу всіх повідомлень. Якщо клієнт швидко надсилає кілька оновлень, сервер може показати лише найновіше. Обмін призначений для процесів одного сеансу Windows на одному комп’ютері.

Посилання на GitHub репозиторій

https://github.com/ArteeCool/CPPLabs/tree/lab-6

Висновок

Реалізовано сервер і два клієнти за варіантом 1 з обміном через проєкцію файлу в пам’ять. М’ютекс узгоджує доступ до даних, подія забезпечує автоматичне оновлення сервера. Перевірено український текст, одночасні записи, закриття та відновлення обміну після повторного запуску. Завдання з автоматичним виявленням змін виконано.